

PREVISÃO DE SÉRIES TEMPORAIS COM REDES NEURAIS RECORRENTES (LSTM)

André Luís Mendes Júnior
Bruna Fontes de Castro
Felipe do Amparo Lauria



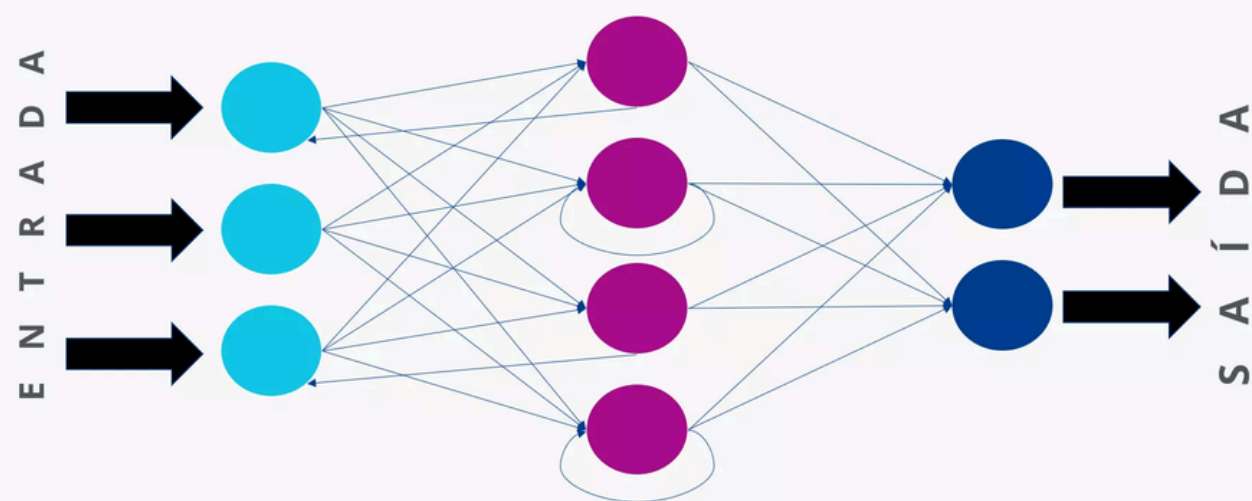
Introdução: Redes Neurais Recorrentes

O que é uma RNN?

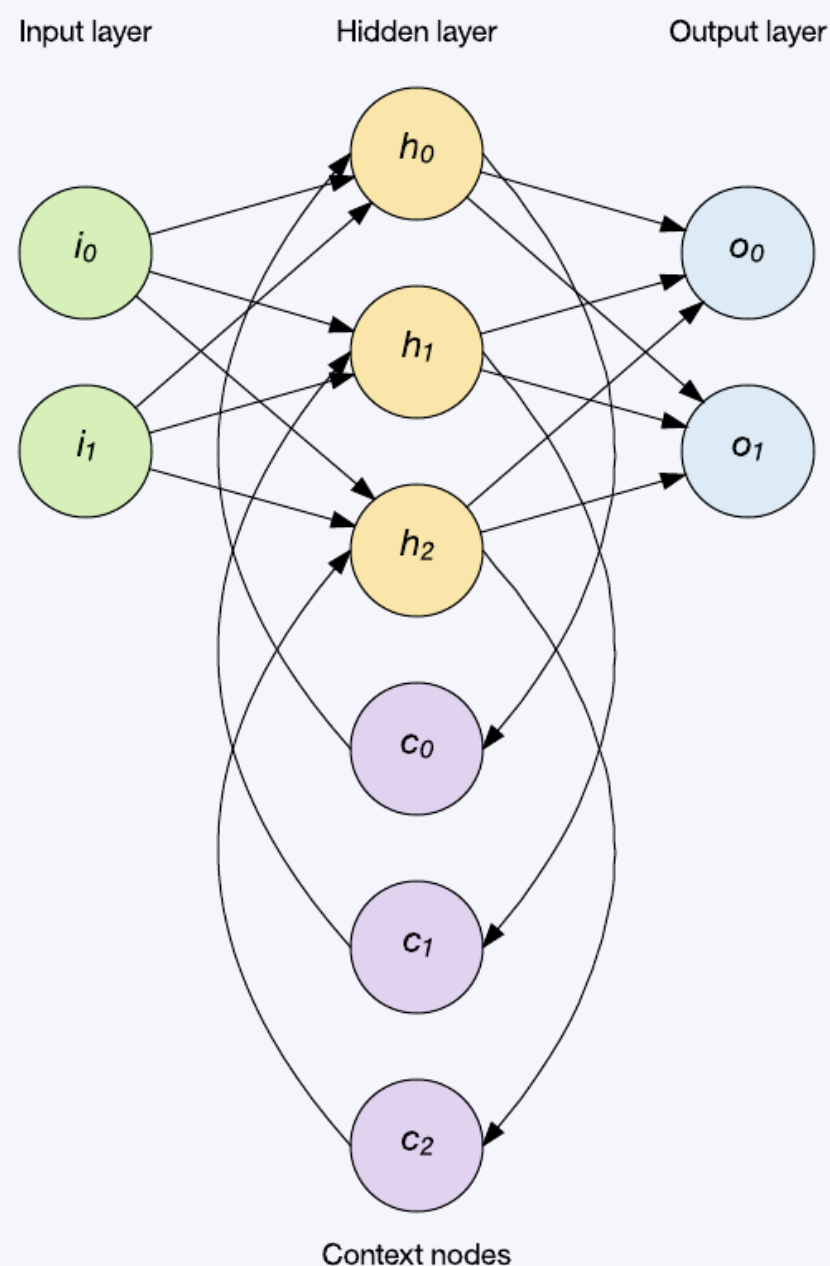
É uma rede neural profunda treinada com dados sequenciais ou séries temporais para criar um modelo de ML capaz de fazer previsões ou tirar conclusões com base em entradas sequenciais.

Enquanto as redes tradicionais de deep learning tratam entradas e saídas como elementos independentes, as redes neurais recorrentes dependem dos elementos anteriores na sequência. Elas utilizam informações dos inputs anteriores para influenciar as entradas e saídas atuais.

Rede Neural Artificial: Recorrente



IONOS



As RNNs funcionam passando os dados sequenciais que recebem para as **camadas ocultas**, uma etapa de cada vez. No entanto, eles também têm um fluxo de trabalho em loop automático ou recorrente: a camada oculta pode lembrar e usar entradas anteriores para previsões futuras em um componente de memória de curto prazo. Ele usa a entrada atual e a memória armazenada para prever a próxima sequência.

Arquitetura LSTM (Long Short-Term Memory)

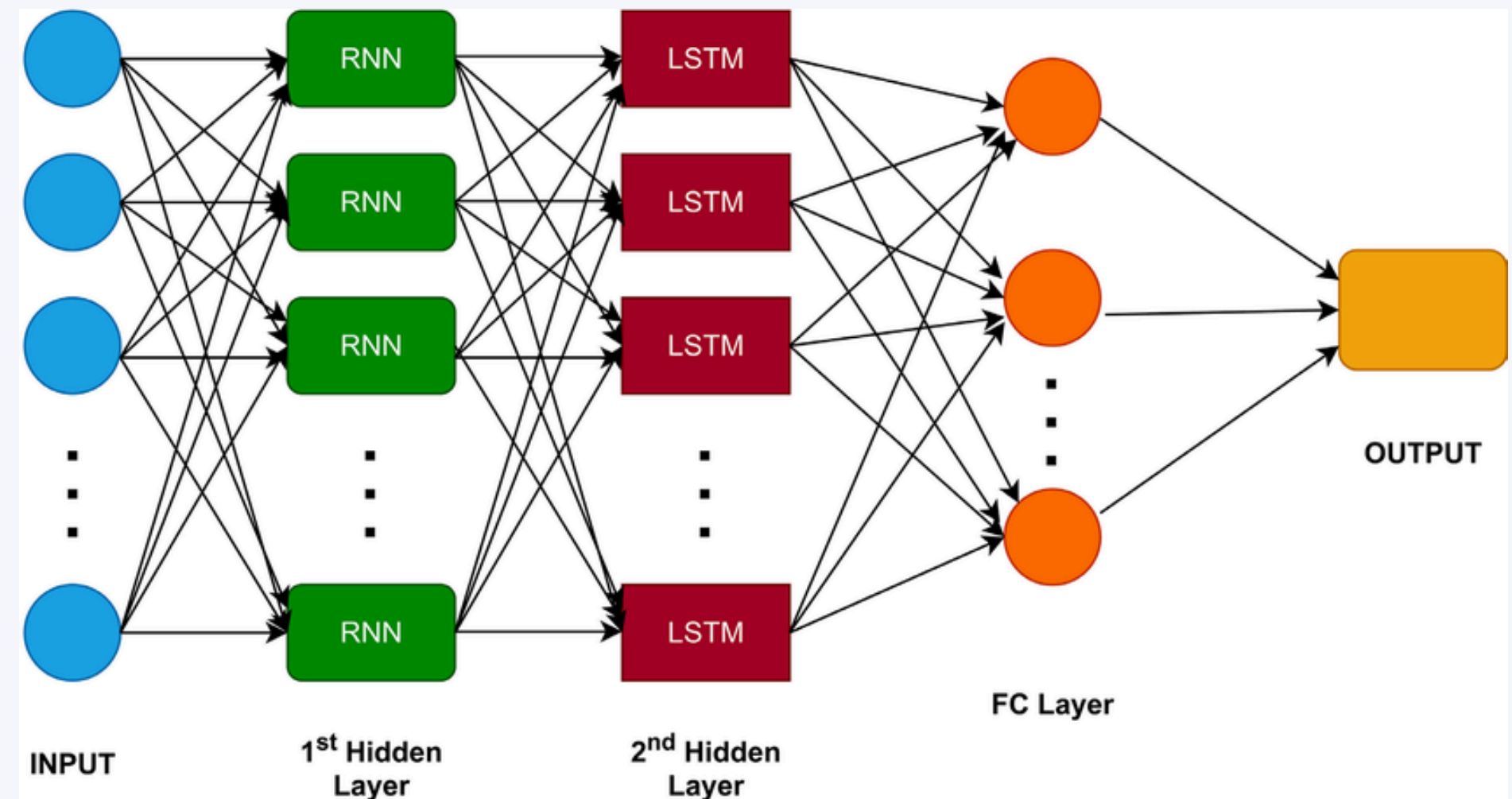
Embora as RNNs tenham memória, essa memória de curto prazo é muito limitada.

Em sequências longas, elas têm dificuldade em reter informações de passos de tempo distantes, esquecendo o "começo" da sequência.

A **Long Short-Term Memory (LSTM)** é uma arquitetura de RNN especializada, projetada para superar essa limitação de "esquecimento."

Enquanto as RNNs tradicionais sofrem com esquecimento, a LSTM introduz a Célula de Estado (Cell State) para reter memórias de longo prazo. Na imagem, esse fluxo de aprendizado profundo ocorre assim:

- **INPUT:** Porta que recebe os dados brutos da série temporal.
- **1ª Camada Oculta:** Realiza o processamento sequencial inicial dos dados.
- **2ª Camada Oculta:** Assume o fluxo anterior, utilizando seus mecanismos internos para filtrar e memorizar os padrões complexos de longo prazo.
- **FC Layer / Camada Densa:** Pega toda a análise temporal abstrata feita pelas LSTMs e combina as informações de forma totalmente conectada.
- **OUTPUT:** É a ponta final onde a camada densa entrega a previsão do preço calculada.



Objetivo

O objetivo principal do projeto foi prever o preço de fechamento diário das ações da empresa Apple Inc. (ticker: AAPL), utilizando dados históricos reais para validar a capacidade preditiva da inteligência artificial frente à volatilidade do mercado financeiro.

Metodologia e Pré-Processamento

1. Coleta de Dados

Utilização da API **yfinance** para extrair o histórico diário de negociação da ação AAPL entre o período de **01/01/2020** a **01/06/2026**.

```
# =====  
# COLETA DOS DADOS  
# =====  
# dados da Apple (AAPL) de 2020-2026  
ticket = "AAPL"  
dados = yf.download(ticket, start="2020-01-01", end="2026-06-01")  
  
dados_fechamento = dados[['Close']].values  
  
print(f"Total de dias baixados: {len(dados_fechamento)}")
```

O foco do modelo foi a variável **Close** (Preço de Fechamento).

2. Divisão Cronológica

A divisão dos dados não foi aleatória, ela respeitou a natureza temporal dos dados (visto que é uma RNN).

```
# =====  
# DIVISÃO DOS DADOS  
# =====  
tamanho_treino = int(len(dados_fechamento) * 0.80)  
  
dados_treino = dados_fechamento[0:tamanho_treino]  
dados_teste = dados_fechamento[tamanho_treino:len(dados_fechamento)]  
  
print(f"Registros para Treino: {len(dados_treino)} | Registros para Teste: {len(dados_teste)}")
```

Os primeiros **80%** da série cronológica foram destinados ao treino do modelo, enquanto os **20%** finais foram estritamente reservados para o conjunto de teste (validação com dados inéditos).

3. Normalização

As funções de ativação desse tipo de rede neural são muito sensíveis à escala dos dados, (funcionam mal com números maiores).

```
# =====  
# NORMALIZAÇÃO  
# =====  
scaler = MinMaxScaler(feature_range=(0, 1))  
  
# só nos dados de treino para evitar vazamento de dados  
dados_treino_normalizados = scaler.fit_transform(dados_treino)  
# mesma transformação no teste  
dados_teste_normalizados = scaler.transform(dados_teste)
```

Para solucionar isso aplicou-se o método **MinMaxScaler**, transformando todos os preços para o intervalo dinâmico entre 0 e 1.

O ajuste (**fit**) foi feito exclusivamente na base de treino.



4. Janelamento Temporal

```
# =====  
# CRIAÇÃO DAS JANELAS TEMPORAIS  
# =====  
def criar_janelas(dataset, tamanho_janela=60):  
    X, Y = [], []  
    for i in range(len(dataset) - tamanho_janela):  
        # do índice 'i' até 'i + tamanho_janela' será a entrada (X)  
        X.append(dataset[i:(i + tamanho_janela), 0])  
        # o próximo elemento imediatamente após a janela será o alvo (Y)  
        Y.append(dataset[i + tamanho_janela, 0])  
    return np.array(X), np.array(Y)  
  
# configurar o tamanho da janela  
tamanho_da_janela = 60  
  
X_treino, y_treino = criar_janelas(dados_treino_normalizados, tamanho_da_janela)  
X_teste, y_teste = criar_janelas(dados_teste_normalizados, tamanho_da_janela)
```

Para estruturar o aprendizado supervisionado,
foi definida uma janela temporal de 60 dias.

Isso significa que, para prever o preço do 61º dia, a rede utilizou como variáveis de entrada (X) o comportamento dos 60 dias imediatamente anteriores.

4. Janelamento Temporal

```
# =====  
# REDIMENSIONAMENTO PARA O PADRÃO DA LSTM (3D)  
# =====  
# mudar os dados em 2d para 3d.  
  
X_treino = np.reshape(X_treino, (X_treino.shape[0], X_treino.shape[1], 1))  
X_teste = np.reshape(X_teste, (X_teste.shape[0], X_teste.shape[1], 1))  
  
print(f"X_treino final: {X_treino.shape}")  
print(f"Formato final do X_teste (Pronto para LSTM): {X_teste.shape}")
```

Os dados foram redimensionados para o formato tridimensional exigido pelo framework Keras:

[Amostras, Passos de Tempo, Recursos] .

Arquitetura da Rede Neural Implementada

Construção Inicial

Construção:

- O modelo foi construído com a biblioteca TensorFlow/Keras, adotando uma abordagem sequencial. A primeira camada é uma LSTM com 50 neurônios.

```
# =====  
# CONSTRUÇÃO DA ARQUITETURA DA LSTM  
# =====  
model = Sequential()  
  
# primeira camada LSTM  
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_treino.shape[1], 1)))  
model.add(Dropout(0.2))
```

Funcionalidade:

- Iniciar o processamento das séries temporais e preparar os dados para aprendizado profundo.

Atributos:

- Configurada com `return_sequences=True` para permitir o empilhamento de redes recorrentes subsequentes.

Regularização e Consolidação

Construção:

- Adição de uma camada de Dropout, seguida por uma segunda camada LSTM com 50 neurônios, e outro Dropout.

```
# primeira camada LSTM
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_treino.shape[1], 1)))
model.add(Dropout(0.2))

# segunda camada LSTM
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))
```

Funcionalidade:

- O Dropout serve para prevenir o overfitting, ou seja, o ajuste excessivo aos dados de treino. A segunda LSTM serve para consolidar a memória sequencial extraída pela camada anterior

Atributos:

- As camadas de Dropout possuem taxa de 20% e a segunda LSTM possui return_sequences=False

Saída e Treinamento

Construção:

- Inserção da Camada de Saída (Dense) e definição dos parâmetros de compilação da rede.

Funcionalidade:

- Realizar a regressão final e entregar o preço predito para o dia seguinte.

Atributos:

- Camada densa linear com 1 única unidade. O treinamento utilizou otimizador Adam, função de perda MSE, rodando por 15 épocas com batch size igual a 32.

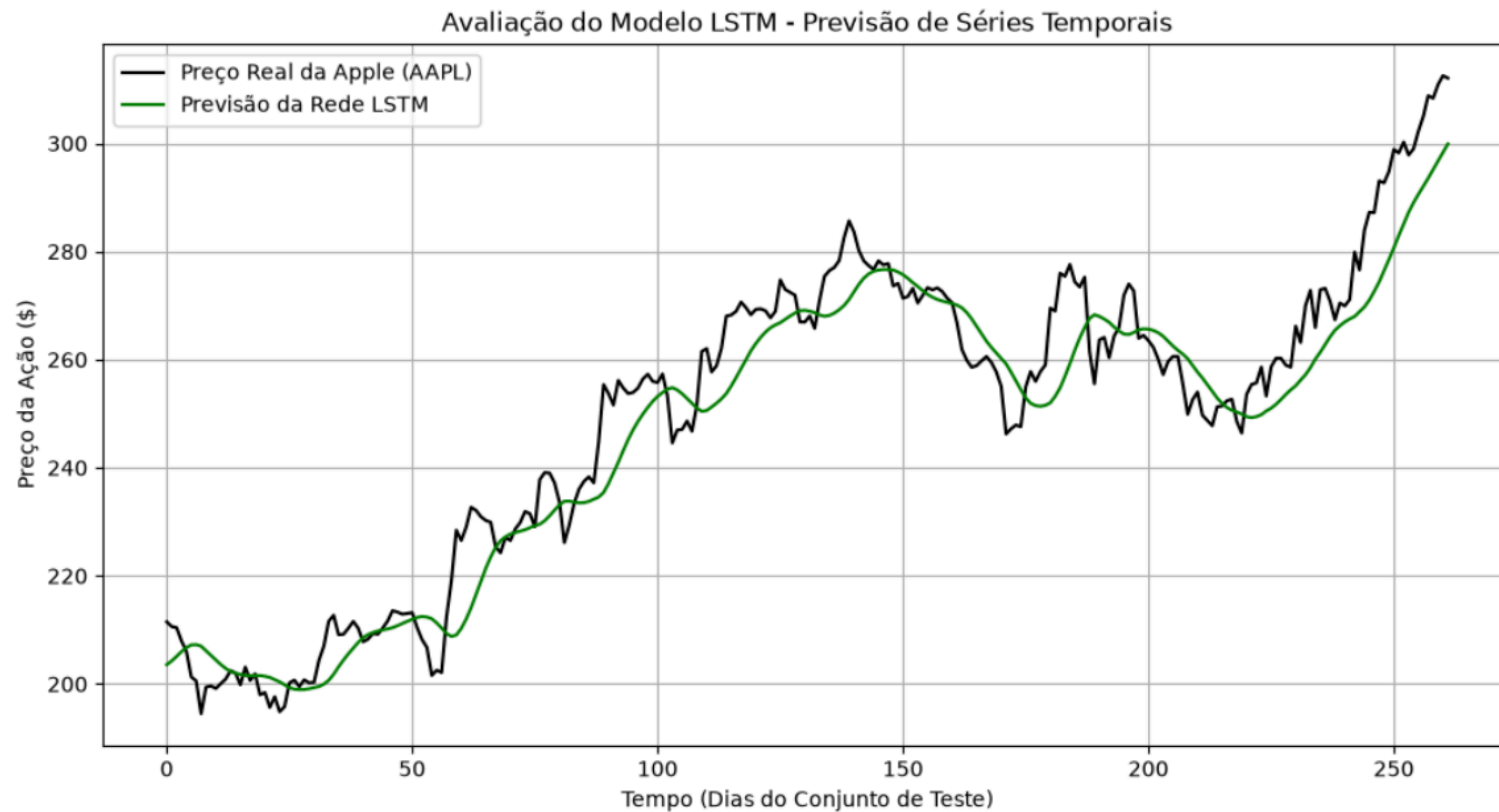
```
# camada de saída (prever o dia seguinte)
model.add(Dense(units=1))

# compilar
model.compile(optimizer='adam', loss='mean_squared_error')

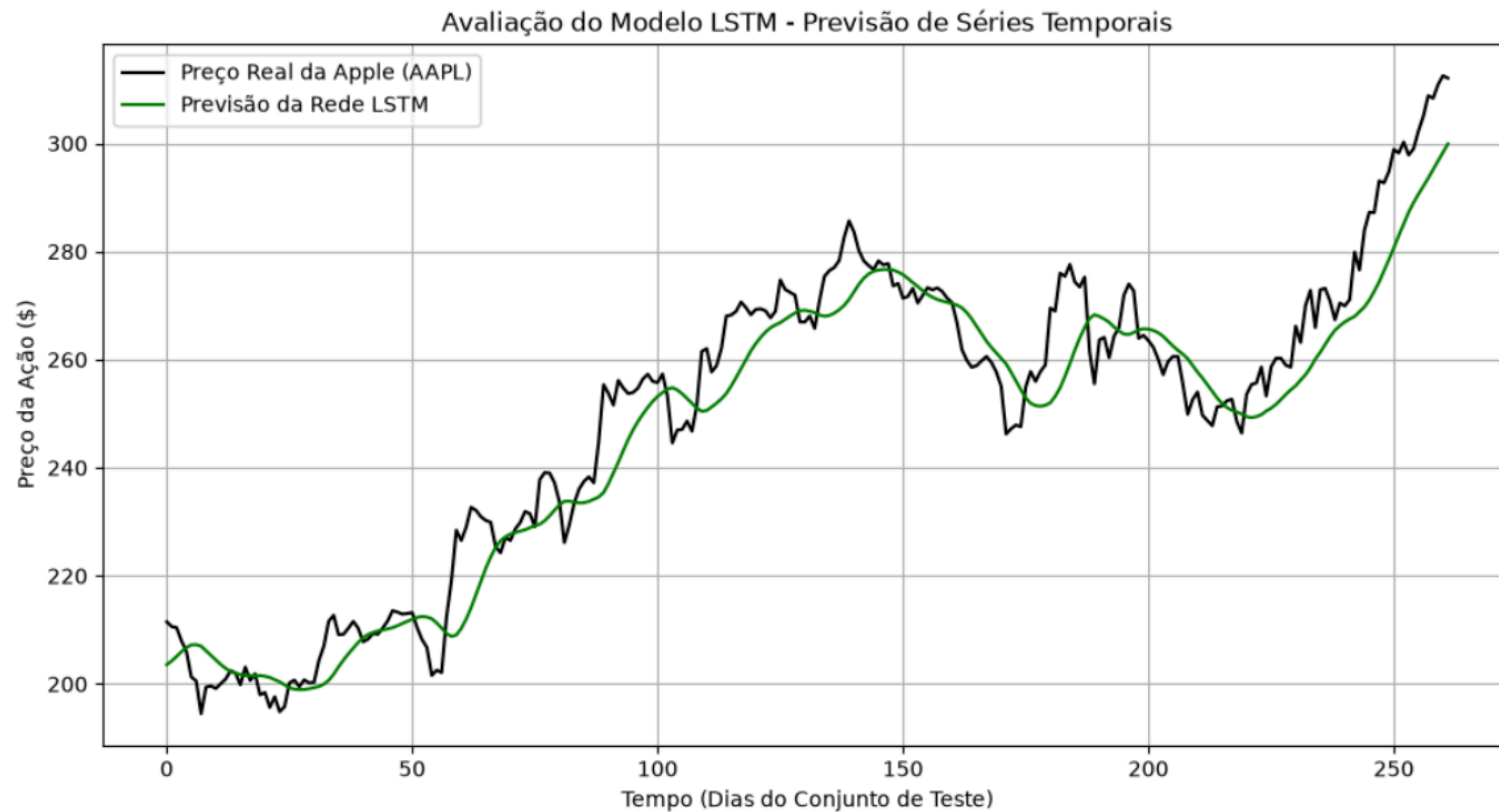
print(model.summary()) # resumo da rede

# =====
# TREINAMENTO
# =====
print("\nIniciando o treinamento")
# epochs = quantas vezes a rede vai olhar todo o dataset
# batch_size = quantos blocos de dados ela olha antes de atualizar os pesos
historico = model.fit(X_treino, y_treino, epochs=15, batch_size=32, validation_split=0.1, verbose=1)
```

Resultados Obtidos



- Após a desnormalização dos dados para a escala real em dólares, observa-se que a curva de previsão (verde) acompanha de forma robusta o comportamento estocástico do preço real (preta)
- A rede conseguiu mapear com precisão os picos de valorização e os vales de desvalorização, resultando em um Coeficiente de Determinação (R^2) excelente de 92,25%



- A curva predita apresenta uma sutil suavização e uma tendência de "perseguição" à curva real, com atraso de um passo temporal.
- Isso ocorre porque o otimizador tenta projetar o preço de amanhã baseando-se fortemente no dia imediatamente anterior para minimizar os erros frente a oscilações abruptas do mercado, o que valida o realismo do experimento.

Considerações Finais

Conclusão

- A arquitetura LSTM demonstrou excelente poder preditivo para prever o fechamento das ações da Apple, superando 92% de precisão explicativa.
- O uso de janelamento temporal associado ao Dropout provou-se altamente eficiente para mitigar os ruídos diários da bolsa.
- O modelo conseguiu focar e extrair perfeitamente a tendência macro do ativo.

Obrigado!